

Coding & Solution

Date	15 March 2026
Team ID	[Your Team ID]
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented code and the overall solution for the Personalized Networking Assistant. The submission demonstrates working functionality across all three core scenarios (smart conversation starter generation, quick fact verification, and history review), clean and modular code practices, and alignment with the defined project requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	[Paste your GitHub / GitLab or hosted link here]
Programming Language(s)	Python 3.11
Framework(s) Used	FastAPI, Streamlit, Hugging Face Transformers (DistilBERT, GPT-2)
Key Features Implemented	Event theme extraction using DistilBERT zero-shot classification; context-aware conversation starter generation using GPT-2; fact verification via the Wikipedia API; conversation history logging; thumbs-up/down feedback capture; interactive Streamlit UI with real-time history and feedback views.
Pending / Incomplete Features	User authentication/login, multi-language support, and cloud deployment are identified as future enhancements beyond the current scope.
Setup / Run Instructions	1) Clone the repository. 2) Create and activate a Python 3.11 virtual environment. 3) Install dependencies with 'pip install -r requirements.txt'. 4) Start the FastAPI backend using 'uvicorn app.main:app --reload'. 5) Launch the Streamlit frontend using 'streamlit run frontend/app.py'. 6) Access the UI at http://localhost:8501 and the API docs at http://localhost:8000/docs .

Code Quality Checklist

S.No	Criteria	Status (Yes/No)
1	Code is modular and organized into functions / classes	Yes

S.No	Criteria	Status (Yes/No)
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

All core modules were unit-tested using pytest, and API endpoints were verified through FastAPI's Swagger UI and httpx-based integration tests prior to local deployment.